



Cyber-Physical Systems
in Mechanical Engineering TU Berlin

Transformers

TU Berlin Summer School 2026 – DL4TS

Michał Brzosko

Cyber-Physical Systems in Mechanical Engineering, Technische Universität Berlin

www.tu.berlin/cpsme

michal.brzosko@tu-berlin.de

Agenda



Cyber-Physical Systems
in Mechanical Engineering TU Berlin

- Shortcomings of RNNs
- The attention mechanism
- The transformer architecture
- Transformer for Time Series

Shortcomings of RNNs



- **Serial computation (recurrence)** preventing parallelization across time steps (training and inference scale linearly with sequence length) $\hat{y}_t = f(x_t, h_{t-1})$
- **Vanishing and exploding gradients:** Learning long range dependencies remains unstable and inefficient
- **Implicit memory addressing** RNNs have **no direct mechanism to access arbitrary past states**.
 - Information retrieval is indirect through repeated state transitions.
→ precise recall of distant tokens unreliable.

- **Summary** of limitations of RNNs:

- computation is serialized
- memory is compressed
- access to the past is indirect

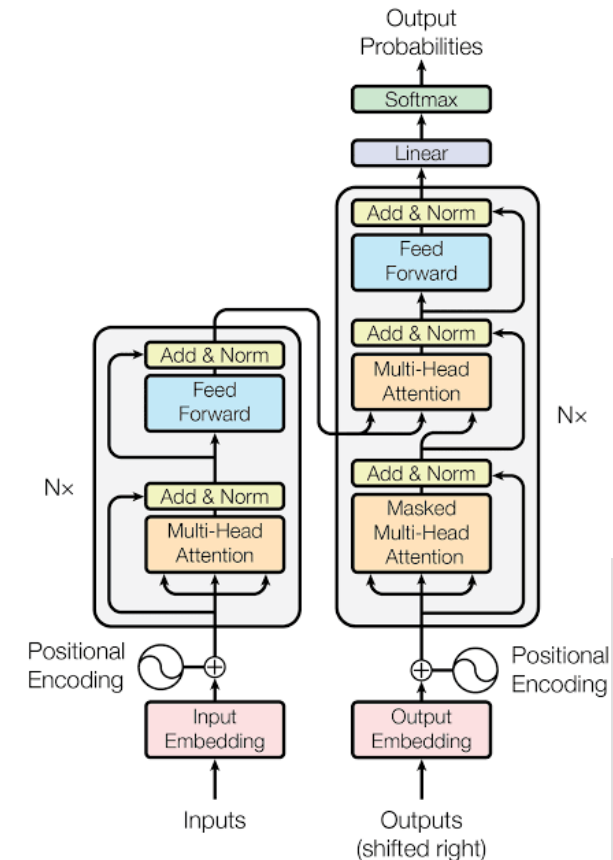
What if we could process the entire sequence at once?

Attention is all you need



- Cited over 250.000 times
- Architecture behind ChatGPT, Claude, Gemini, DeepSeek, Perplexity, ...

The Transformer Architecture



A. Vaswani et al. 2017



Tokens and Embeddings

- **Example:** next character prediction in sequences. Given {"A", "B", "C"}, predict the next character
- 1. **Vocabulary construction:** define a finite set of allowed characters. $V = \{A, B, C, D\}$, $|V| = 4$
- 2. Assign each character a **unique integer id ("token")** $t_i \in \{0, \dots, |V| - 1\}$
 - here: $A \rightarrow 0$, $B \rightarrow 1$, $C \rightarrow 2$, $D \rightarrow 3$
 - example: text sequence "ABACC" $\rightarrow$ tokenized sequence $[0, 1, 0, 2, 2]$
- **Tokenization:** deterministic preprocessing step that converts raw input into a sequence of discrete indices from a finite set.

What tokenization decides:

- how the input is segmented
- what the vocabulary is
- how symbols map to integer ids

Raw text "playing"

Possible tokenizations

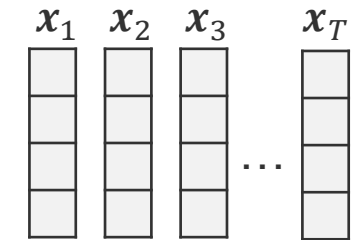
- ["playing"]
- ["play", "ing"]
- ["p", "l", "a", "y", "i", "n", "g"]

Embedding



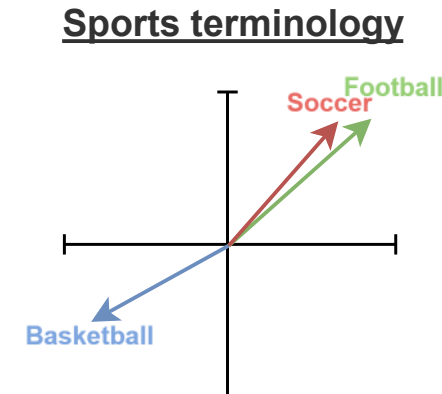
- **Embedding** converts discrete token IDs into continuous vectors that the model can compute over
 - embedding is a learned lookup table $E \in \mathbb{R}^{|V| \times d_{\text{model}}}$ of fixed size
 - embedding vector obtained by querying the lookup table $x_i = E[t_i]$

- Example: 2-dimensional embedding space $x_A = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, $x_B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$, $x_C = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, $x_D = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$



- Embedding of {"A", "B", "C"}: $X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \in \mathbb{R}^{(T \times d_{\text{model}})}$

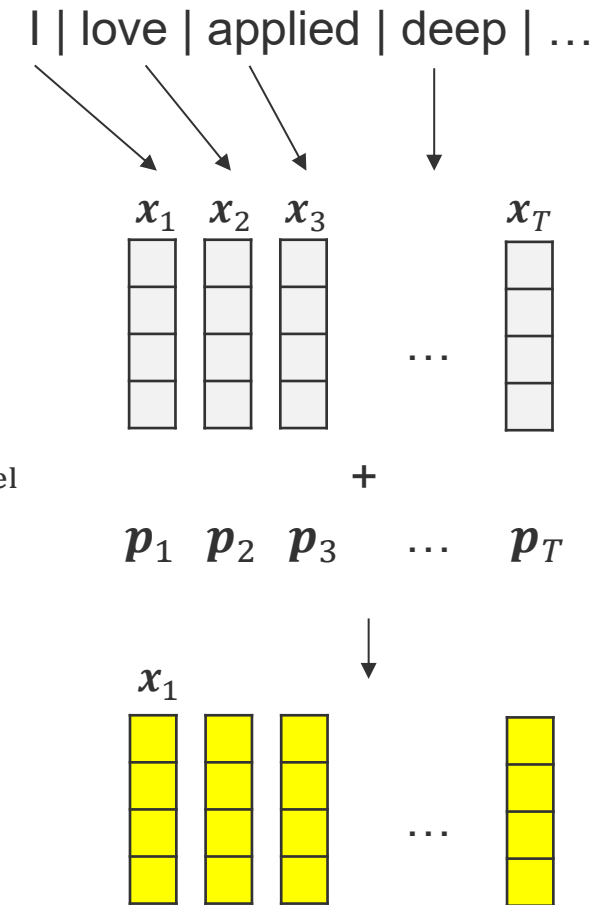
- **Meaning of embeddings**
 - Similar meaning $\rightarrow$ nearby embeddings
 - Distance can reflect semantic similarity



Positional Encoding



- Input: sequence of elements, tokenized and embedded
- Goal:** we want to process that input all at once, and not sequentially
 - order of embedding vectors is not explicitly considered
 - we need to encode **position information** to understand order
- Position-aware encodings** $x_i = x_i + p_i$
 - Position vector, same dimensionality as token embedding $p_i \in \mathbb{R}^{d_{\text{model}}}$
- Methods for representing positional information:
 - Fixed** sinusoidal positional encoding:
$$PE_{i,2k} = \sin\left(\frac{i}{10000^{2k/d_{\text{model}}}}\right),$$
$$PE_{i,2k+1} = \cos\left(\frac{i}{10000^{2k/d_{\text{model}}}}\right), k = 0, \dots, \frac{d_{\text{model}}}{2} - 1$$
 - Learned** positional embedding



Summary



- Critical separation:
 - Tokenization: discrete, symbolic, rule based
 - Embedding: continuous, learned
- Model operates only after tokenization
 - The model never sees raw text.
 - It never decides how text is split.
 - It only consumes tokens' representations.
- Tokenization bounds the problem by defining a finite alphabet.
- Key constraint: tokenization fixes the information interface.
- Sequence order is not inherent and must be added
 - Positional encoding: fixed
 - Positional embedding: learned

Process:

Raw data

→ tokenization

→ tokens

→ embedding lookup

→ sequence of vectors

→ positional encoding

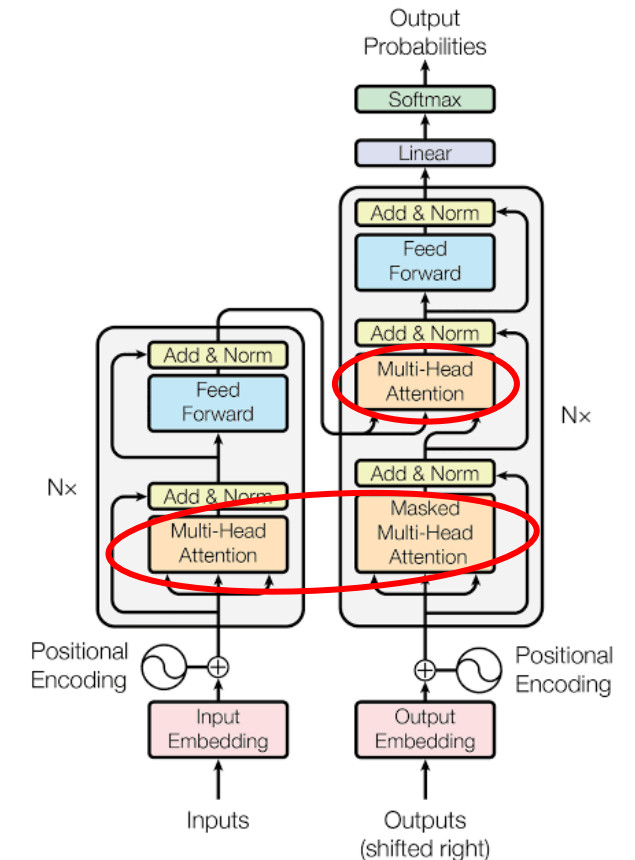
→ model computation

$$\text{attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

The Transformer Architecture

Attention Mechanism

Vaswani et al. (2017): Attention Is All You Need
<https://arxiv.org/abs/1706.03762>



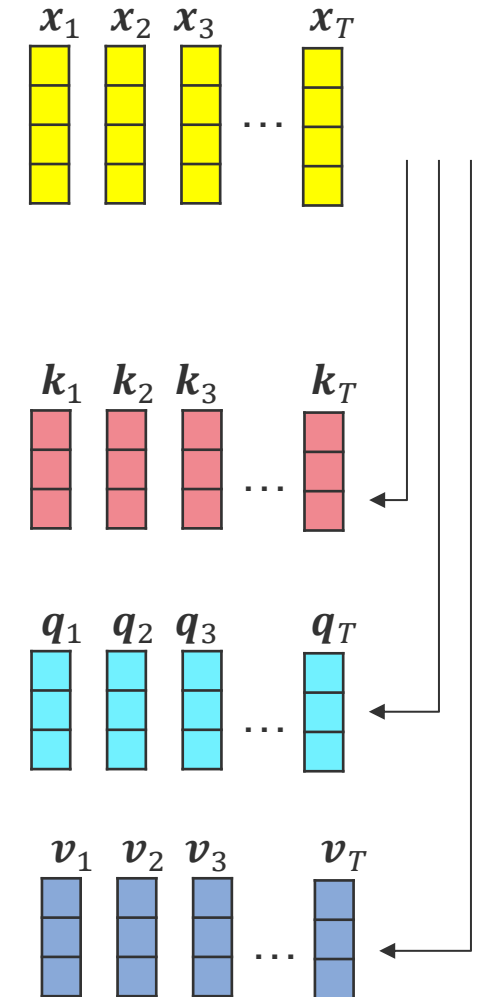
Attention Mechanism



- Sequential input data (e.g. embeddings) $X = [\mathbf{x}_1, \dots, \mathbf{x}_T] \in \mathbb{R}^{(T \times d_{\text{model}})}$
- Three representations are derived:
 - Queries** $Q = XW_Q \in \mathbb{R}^{(T \times d_k)}$, for $W_Q \in \mathbb{R}^{(d_{\text{model}} \times d_k)}$
 - Keys** $K = XW_K \in \mathbb{R}^{(T \times d_k)}$, for $W_K \in \mathbb{R}^{(d_{\text{model}} \times d_k)}$
 - Values** $V = XW_V \in \mathbb{R}^{(T \times d_v)}$, for $W_V \in \mathbb{R}^{(d_{\text{model}} \times d_v)}$

Intuition: Information retrieval systems

- Queries:** What information is this token looking for?
- Keys:** What kind of information does this token contain?
- Values:** What information does this token provide?



Attention Mechanism



- Sequential input data (e.g. embeddings) $X = [\mathbf{x}_1, \dots, \mathbf{x}_T] \in \mathbb{R}^{(T \times d_{\text{model}})}$
- Three representations are derived:
 - Queries** $Q = XW_Q \in \mathbb{R}^{(T \times d_k)}$, for $W_Q \in \mathbb{R}^{(d_{\text{model}} \times d_k)}$
 - Keys** $K = XW_K \in \mathbb{R}^{(T \times d_k)}$, for $W_K \in \mathbb{R}^{(d_{\text{model}} \times d_k)}$
 - Values** $V = XW_V \in \mathbb{R}^{(T \times d_v)}$, for $W_V \in \mathbb{R}^{(d_{\text{model}} \times d_v)}$

- Example** (ABC sequence) {"A", "B", "C"}: $X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$

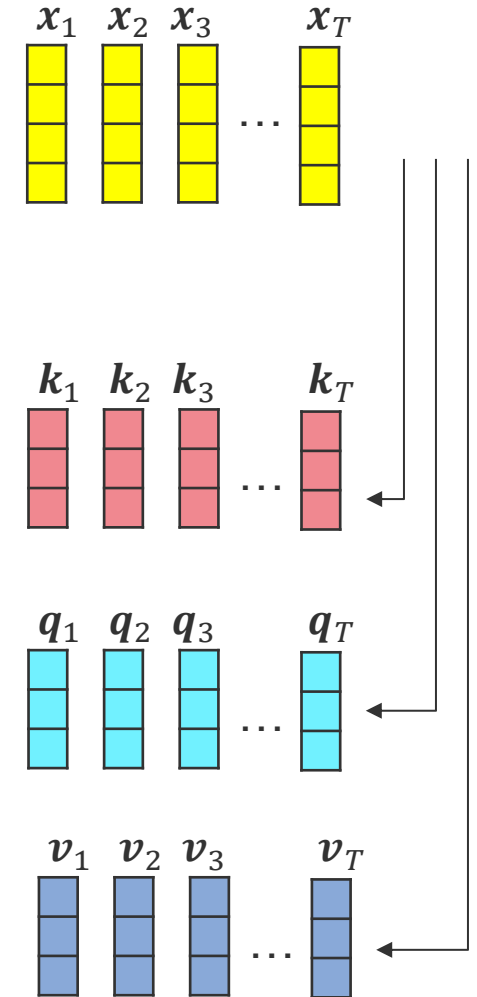
- For weight matrices (for $d_{k,v} = 2$) $W_Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$, $W_K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$, $W_V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

- Here: (weights as identity) $Q = XW_Q = K = V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \in \mathbb{R}^{(T \times d_k)}$

"A"

"B"

"C"



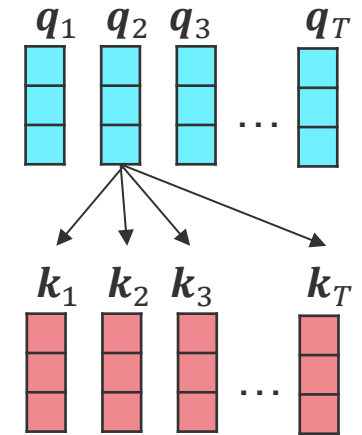
Attention Mechanism



- Pairwise similarity/**attention scores**: attention measures how much each query matches each key.

$$S_{\text{scaled}} = \frac{QK^T}{\sqrt{d_k}} \in \mathbb{R}^{(T \times T)} \text{ for } Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$$

- Scaling: large dot products destabilize gradients $\rightarrow$ scale by dimension d_k .



Example:

$$S_{\text{scaled}} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}$$

"A" "B" "C"

"A"
"B"
"C"

matrix of dot products

$q_2 K^T$

*Which parts of the input
relate to each other?*

*Which parts are
important to each other?*

Attention Mechanism



- **Normalization:** convert scores into **attention weights**

$$A = \text{softmax}(S_{\text{scaled}}) \in \mathbb{R}^{(T \times T)}$$

(across rows, such that the row-sum is 1)

$$S_{\text{scaled}} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}$$

$$A \approx \begin{array}{ccc|c} \text{"A"} & \text{"B"} & \text{"C"} & \\ \hline 0.4 & 0.2 & 0.4 & \text{"A"} \\ 0.2 & 0.4 & 0.4 & \text{"B"} \\ 0.25 & 0.25 & 0.5 & \text{"C"} \end{array}$$

Each row: attention probability distribution over the sequence

Interpretation: similar (basketball, football, tennisball) or related elements (basketball, Nowitzki, referee) are going to have higher pairwise attention weights.

Why? Because their embedding vectors will be more similar

Attention amplifies existing similarity



A is not symmetric because

- Q and K are not equal
- softmax breaks symmetry

Attention Mechanism



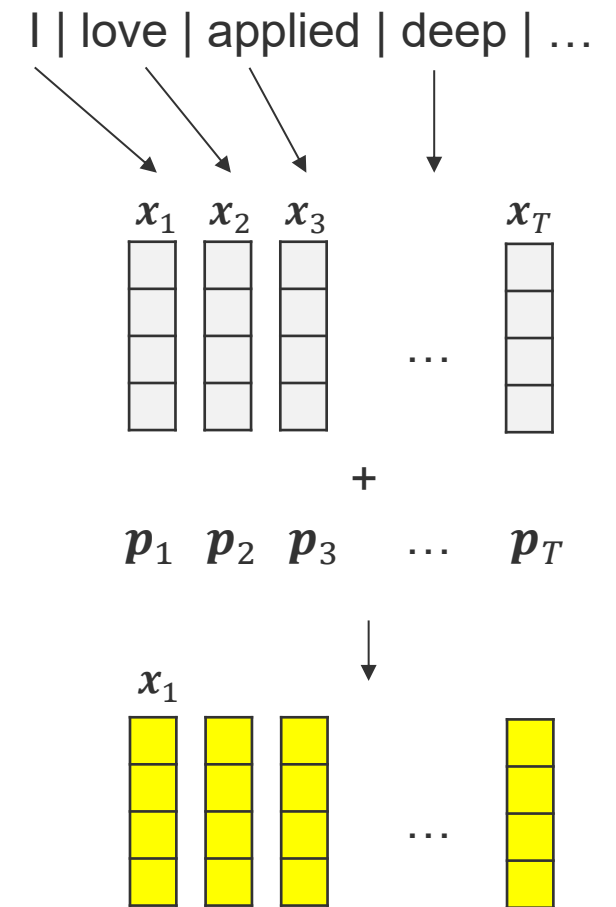
- **Attention** works on **embedded vectors**.
 - Measures similarity
 - Does not encode position

- **Issue:** order matters in sequences

*The food was not **good**, it smelled **bad**.*

←-----→

- **Attention** works on **position-encoded vectors**.
 - allows the model to relate different parts of a sequence to each other when computing a representation for the sequence
 - aggregates information from the entire sequence.
 - each token attends to every other token in the sequence



Attention Mechanism

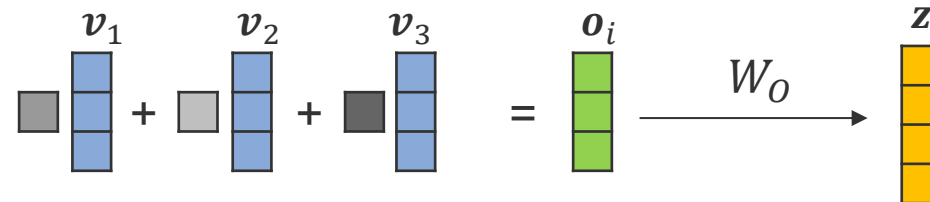


- **Output computation:** extract features with high attention

$$O = AV \in \mathbb{R}^{(T \times d_v)}$$

$$O = \begin{bmatrix} 0.4 & 0.2 & 0.4 \\ 0.2 & 0.4 & 0.4 \\ 0.25 & 0.25 & 0.5 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 0.80 & 0.60 \\ 0.60 & 0.80 \\ 0.75 & 0.75 \end{bmatrix} \begin{matrix} o_0 \\ o_1 \\ o_2 \end{matrix}$$


- Each output vector o_i is an attention-weighted sum of all value vectors



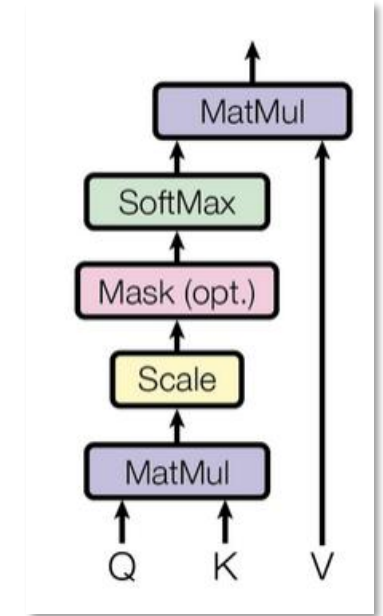
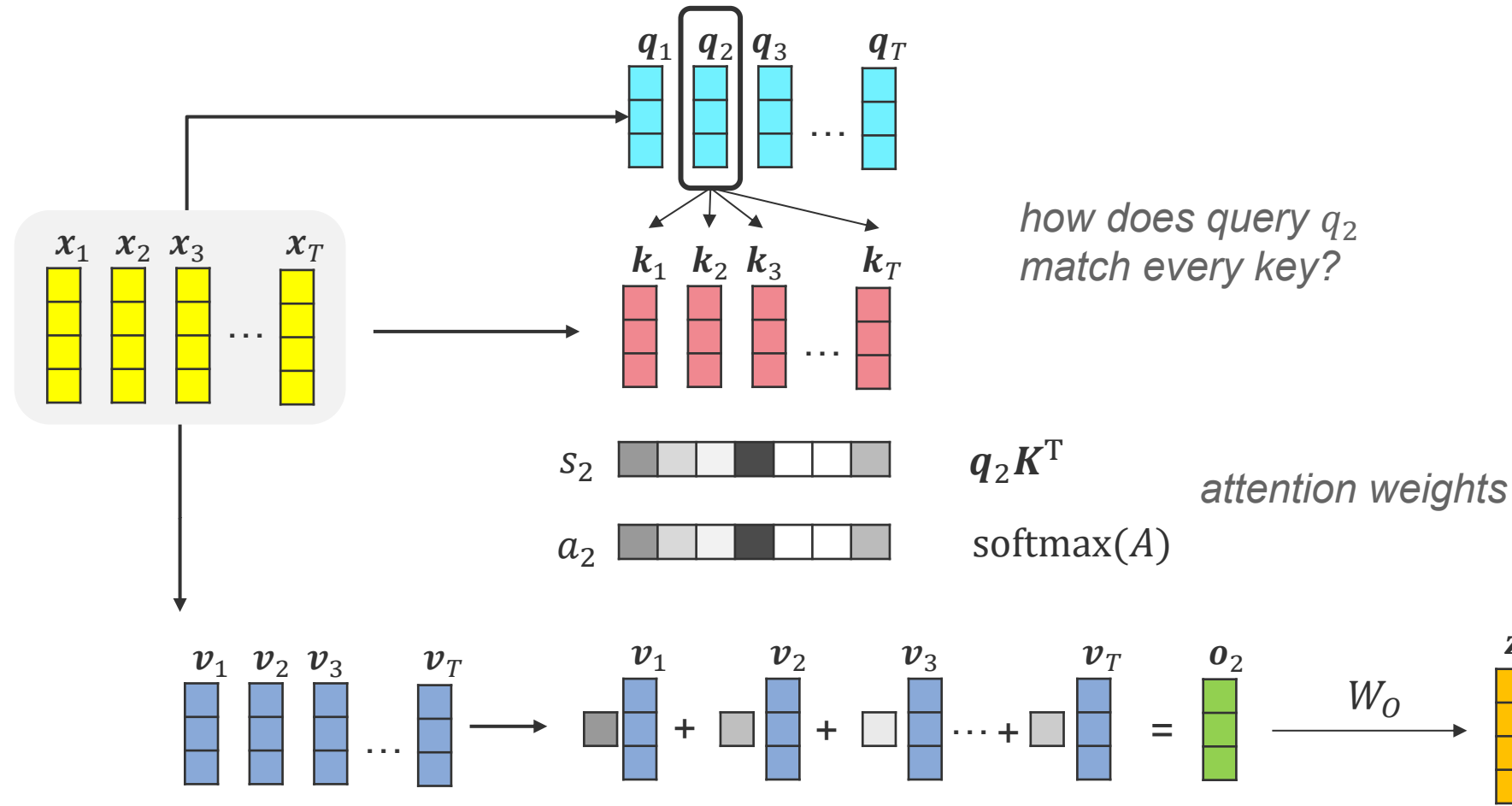
- **Output projection:** project back to embedding dimension through learnable W_O

$$Z = OW_O \in \mathbb{R}^{(T \times d_{\text{model}})}, W_O \in \mathbb{R}^{(d_v \times d_{\text{model}})}$$

→ context vectors z_i

Overview

$$\text{attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Attention Mechanism



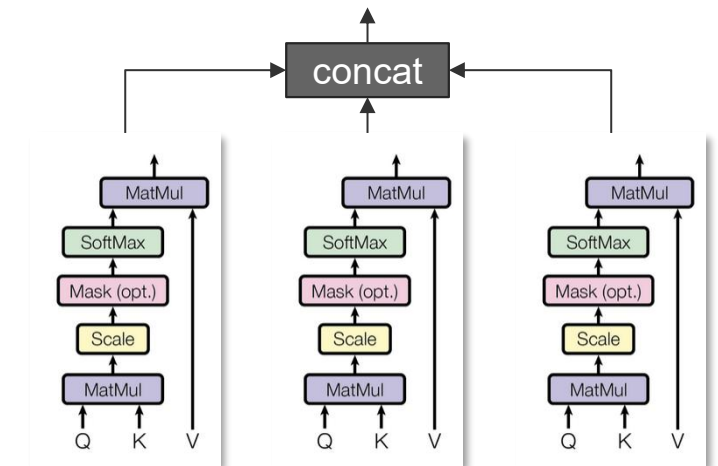
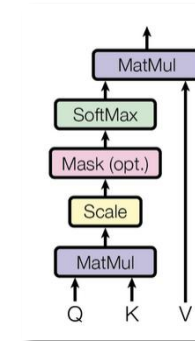
- Shapes and dimensions:

- **Single head attention:** $d_k = d_v = d_{\text{model}}$

- **Multi head attention** (H heads): $d_k = d_v = \frac{d_{\text{model}}}{H}$

Concatenated output: $\text{concat}(\text{head}_1, \dots, \text{head}_H) \in \mathbb{R}^{(T \times d_{\text{model}})}$

- d_k : **dimension of queries and keys:** size of the compatibility space for measuring relationships between elements
- d_v : **dimension of values:** size of the information being passed along in the attention mechanism

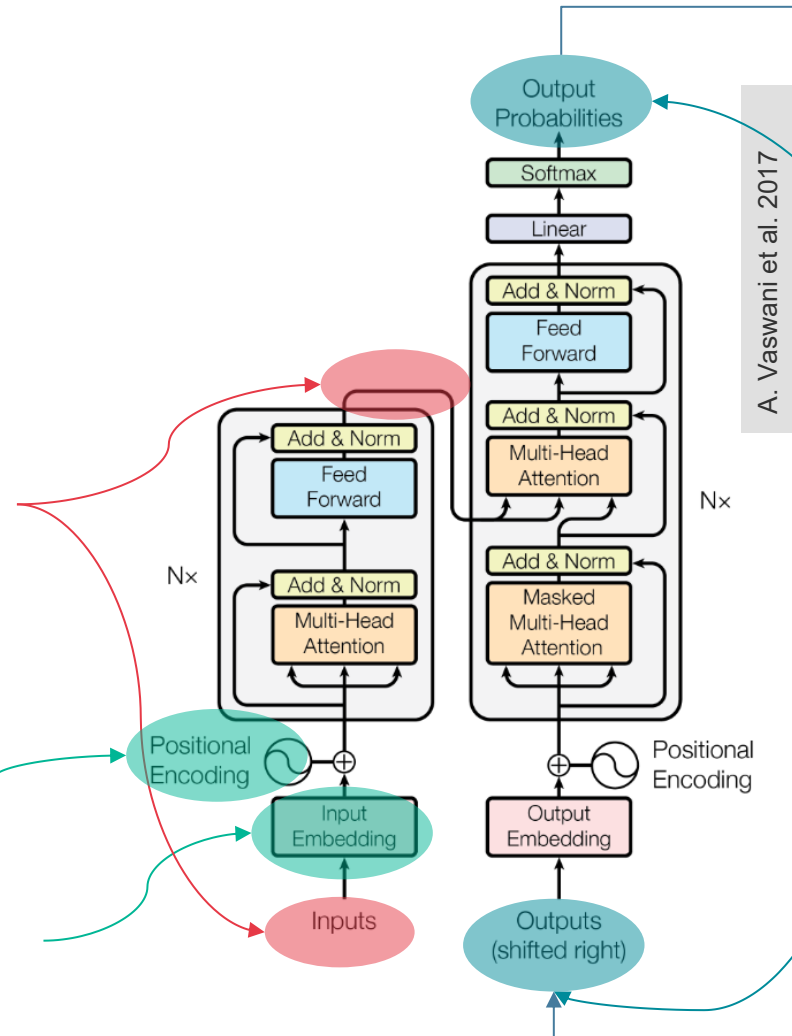


Transformer Architecture



Encoder: maps input sequence to a continuous latent representation. Holds the information for the entire input sequence

Positional encoding
Word embedding layer: words $\rightarrow$ vec



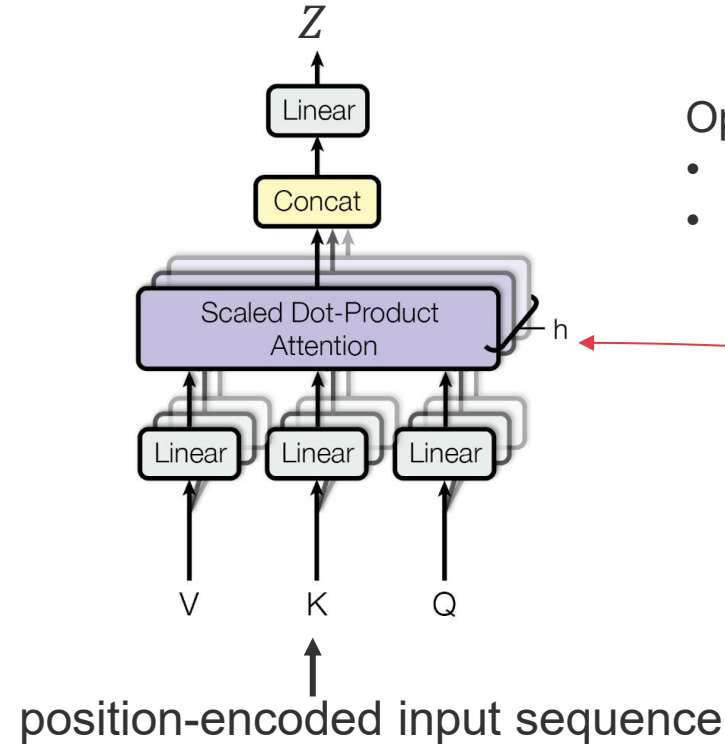
Decoder: step-by-step generates an output from the continuous representation and its previous outputs

Transformer Architecture



▪ **Encoder** and multi-headed self-attention

Attention weights for each input token: how should each word attend to the other words in the sequence?



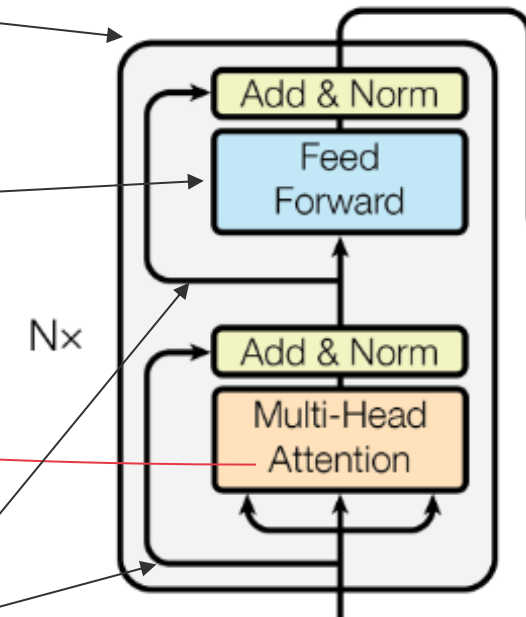
continuous input representation
with attention information

Transformer
encoder layer

Operates independently per token

- increase representational power
- perform feature transformation

residual
connections

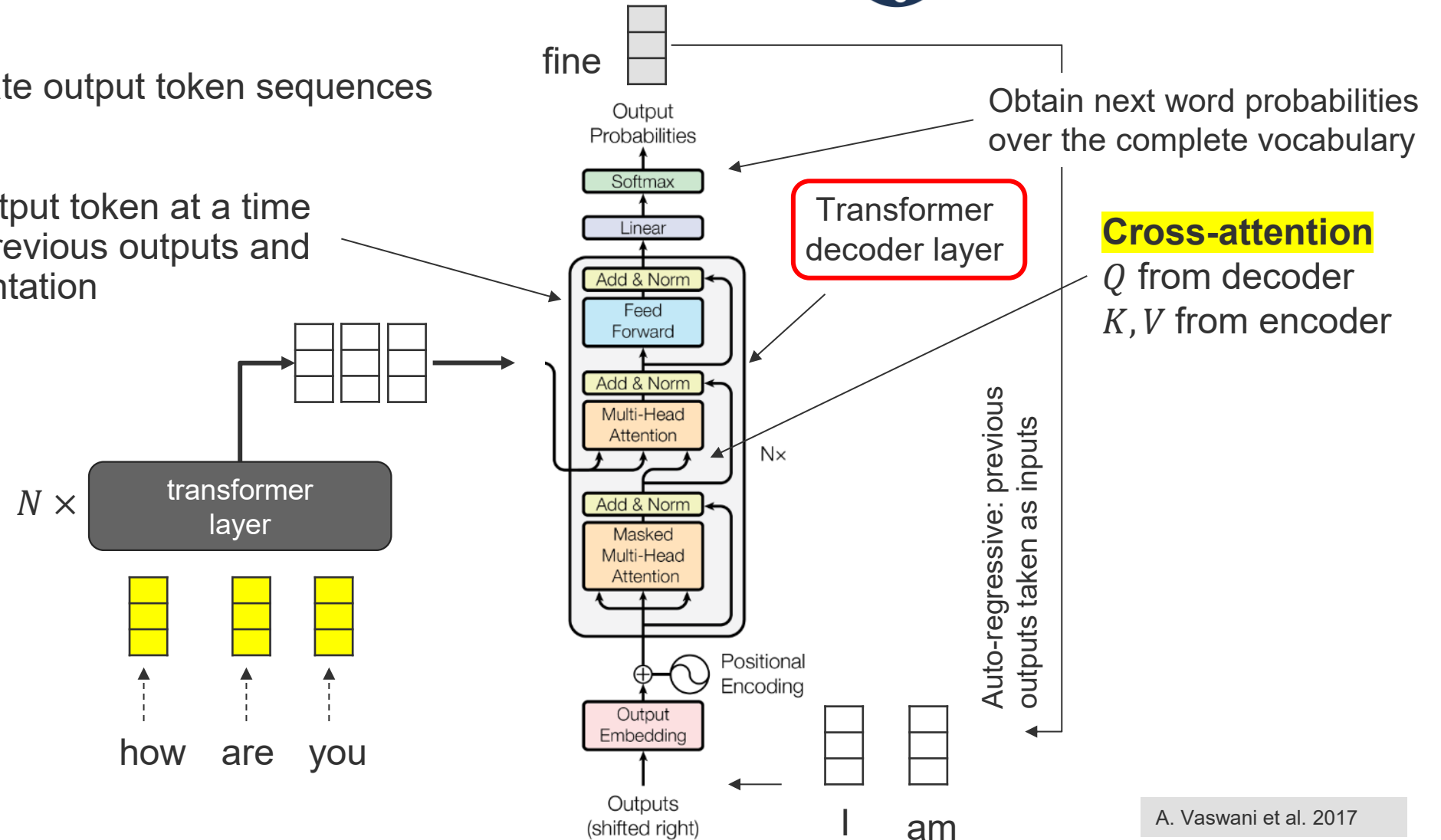


A. Vaswani et al. 2017

Transformer Architecture



- **Decoder:** generate output token sequences
- Produces one output token at a time conditioned on previous outputs and encoder representation



A. Vaswani et al. 2017

Transformers: Overview



Cyber-Physical Systems
in Mechanical Engineering TU Berlin

- Architecture integrates seamlessly with highly parallelizable compute hardware (GPUs)
- **One architecture, many tasks:** language translation, video captioning, sentiment analysis, time series prediction, ...
- **Ability to capture long-term dependencies** irrespective of input order.
- **Quadratic scaling** with context length! $\rightarrow A = \text{softmax}(S_{\text{scaled}}) \in \mathbb{R}^{(T \times T)}$
- State-of-the-art architecture for many tasks since ~2018.

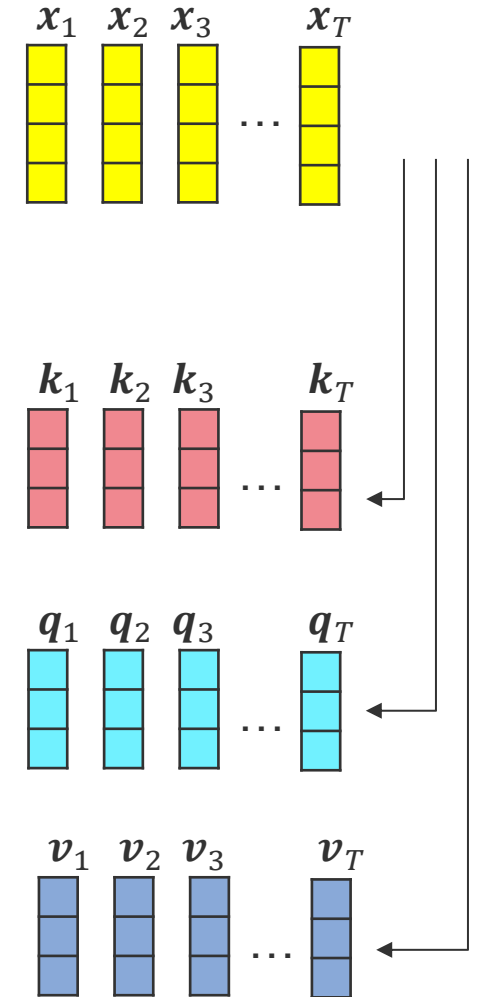


Transformers for Time Series

Transformers for Time Series



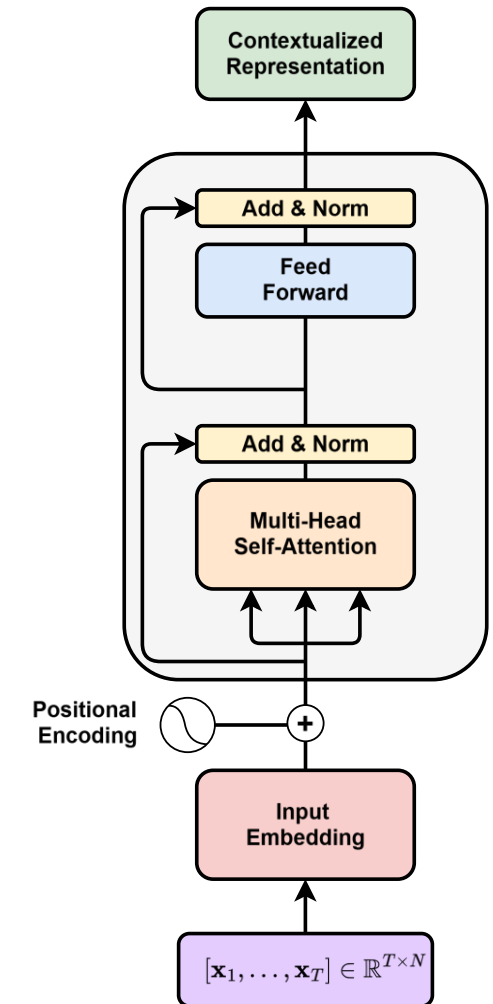
- Sequential input data (e.g. embeddings) $X = [\mathbf{x}_1, \dots, \mathbf{x}_T] \in \mathbb{R}^{(T \times d_{\text{model}})}$
- Three representations are derived:
 - Queries** $Q = XW_Q \in \mathbb{R}^{(T \times d_k)}$, for $W_Q \in \mathbb{R}^{(d_{\text{model}} \times d_k)}$
 - Keys** $K = XW_K \in \mathbb{R}^{(T \times d_k)}$, for $W_K \in \mathbb{R}^{(d_{\text{model}} \times d_k)}$
 - Values** $V = XW_V \in \mathbb{R}^{(T \times d_v)}$, for $W_V \in \mathbb{R}^{(d_{\text{model}} \times d_v)}$
- Time series**
 - Univariate: $x = [\mathbf{x}_1, \dots, \mathbf{x}_T] \in \mathbb{R}^{(T \times 1)}$
 - Multivariate: $X = [\mathbf{x}_1, \dots, \mathbf{x}_T] \in \mathbb{R}^{(T \times N)}$
 - Embedding of the time series to $[\mathbf{x}_1, \dots, \mathbf{x}_T] \in \mathbb{R}^{(T \times d_{\text{model}})}$
 - Add positional encoding
 - Results in a sequence embedded vectors



Encoder-Only Transformer



- **Input:** $X = [x_1, \dots, x_T] \in \mathbb{R}^{(T \times N)}$
- **Project to model dimension:** $X \in \mathbb{R}^{(T \times d_{\text{model}})}$
 - Map the feature dimension from N to d_{model}
 - e.g. via a linear projection
- **Contextualized representation:** $Z \in \mathbb{R}^{(T \times d_{\text{model}})}$
- **Task-specific processing**
 - Seq2Seq regression
 - Classification
 - Forecasting



Summarizing Time Series



- **Contextualized representation:** $Z \in \mathbb{R}^{(T \times d_{\text{model}})}$

1. Map the contextualized representation to a scalar output

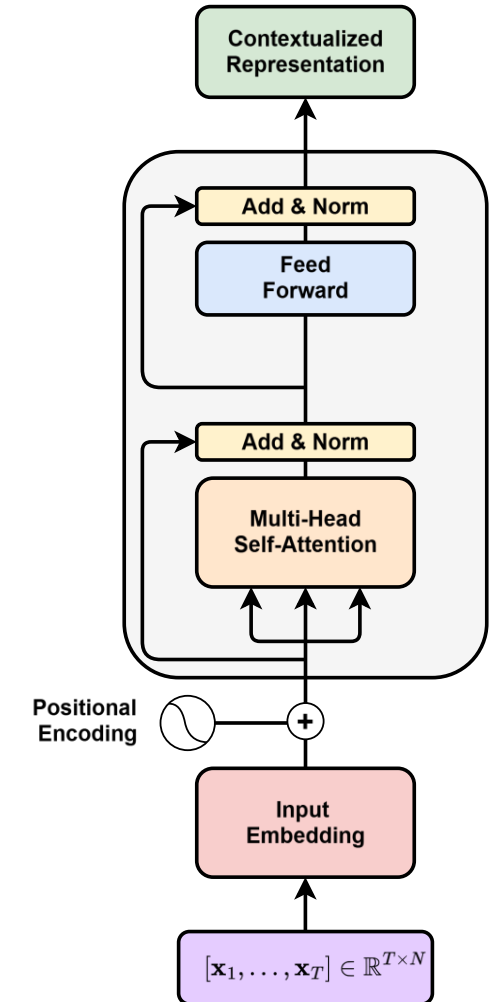
- Flatten $Z \in \mathbb{R}^{(T \times d_{\text{model}})} \rightarrow z_{\text{flat}} \in \mathbb{R}^{(T * d_{\text{model}} \times 1)}$
- e.g. $T = 500$, $d_{\text{model}} = 64 \rightarrow \underline{32001}$ trainable parameters

2. Mean or max pooling

- $z_{\text{mean}} = \frac{1}{T} \sum_{t=1}^T z_t \in \mathbb{R}^{d_{\text{model}}}$ or $z_{\text{max}} \in \mathbb{R}^{d_{\text{model}}}$
- e.g. $T = 500$, $d_{\text{model}} = 64 \rightarrow \underline{65}$ trainable parameters

3. CLS Token

- Append a CLS Token to the start of the sequence
- e.g. $T = 500$, $d_{\text{model}} = 64 \rightarrow \underline{64 + 65 = 129}$ trainable parameters



Summarizing Time Series



- **Contextualized representation:** $Z \in \mathbb{R}^{(T \times d_{\text{model}})}$

1. Map the contextualized representation to a scalar output

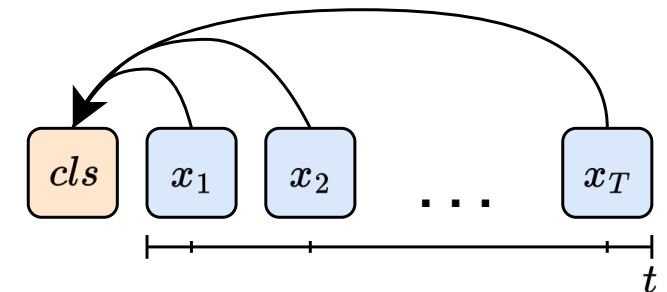
- Flatten $Z \in \mathbb{R}^{(T \times d_{\text{model}})} \rightarrow z_{\text{flat}} \in \mathbb{R}^{(T * d_{\text{model}} \times 1)}$
- e.g. $T = 500$, $d_{\text{model}} = 64 \rightarrow \underline{32001}$ trainable parameters

2. Mean or max pooling

- $z_{\text{mean}} = \frac{1}{T} \sum_{t=1}^T z_t \in \mathbb{R}^{d_{\text{model}}}$ or $z_{\text{max}} \in \mathbb{R}^{d_{\text{model}}}$
- e.g. $T = 500$, $d_{\text{model}} = 64 \rightarrow \underline{65}$ trainable parameters

3. CLS Token

- Append a CLS Token to the start of the sequence
- e.g. $T = 500$, $d_{\text{model}} = 64 \rightarrow \underline{64 + 65 = 129}$ trainable parameters



Decoder-Only Transformer



Main setting: Autoregressive forecasting

- Given an observed history: $[\mathbf{y}_1, \dots, \mathbf{y}_{T-1}] \in \mathbb{R}^{(T-1 \times N)}$
- The model predicts the next time step: $\hat{\mathbf{y}}_T = \mathbf{f}(\mathbf{y}_1, \dots, \mathbf{y}_{T-1}) \in \mathbb{R}^{(1 \times N)}$
- Append generated output to the input sequence and iterate

▪ Causal (Masked) self-attention:

- Each token may attend only to itself and previous tokens
- Attention to future positions is blocked
- Prevents information leakage

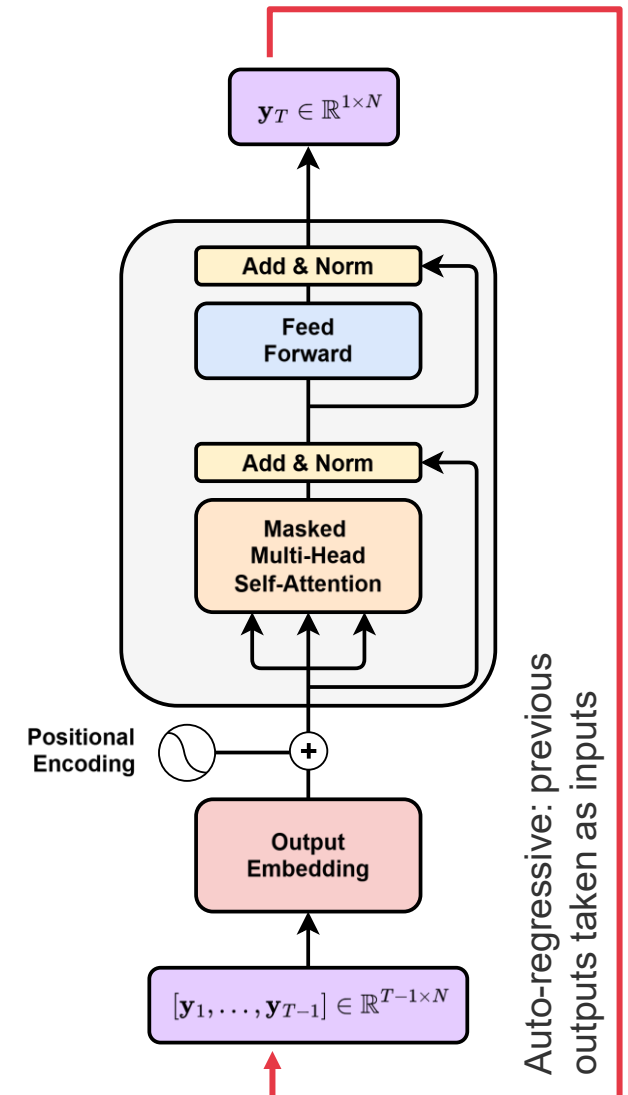
Causal Mask

$$M = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

▪ Autoregressive generation of the output

▪ Context strategies:

- Growing context $[\mathbf{y}_1, \mathbf{y}_2, \mathbf{y}_3] \rightarrow [\mathbf{y}_1, \mathbf{y}_2, \mathbf{y}_3, \mathbf{y}_4]$
- Sliding context $[\mathbf{y}_1, \mathbf{y}_2, \mathbf{y}_3] \rightarrow [\mathbf{y}_2, \mathbf{y}_3, \mathbf{y}_4]$



Encoder-Decoder Transformer



Main setting: Seq2Seq regression

- **Input sequence:** $X = [x_1, \dots, x_T] \in \mathbb{R}^{(T_x \times N_x)}$
- **Output sequence:** $Y = [y_1, \dots, y_T] \in \mathbb{R}^{(T_y \times N_y)}$ (Seq2Seq regression)

- **Cross-Attention:**

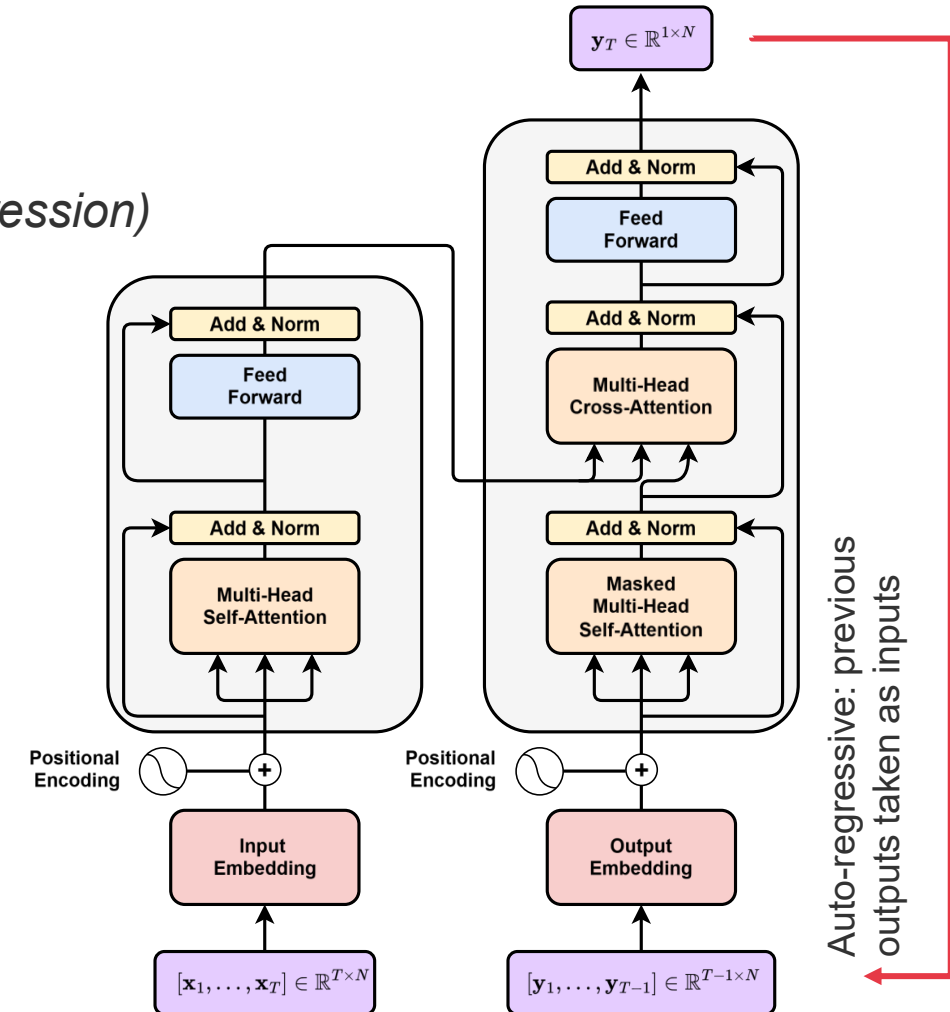
- Decoder states act as **queries**
- Encoder states provide **keys** and **values**

$$Q = Z_{dec}, K = V = Z_{enc} \rightarrow A_{cross} \in \mathbb{R}^{(T_y \times T_x)}$$

- **Autoregressive generation of the output**

- **Context strategies:**

- Growing context $[y_1, y_2, y_3] \rightarrow [y_1, y_2, y_3, y_4]$
- Sliding context $[y_1, y_2, y_3] \rightarrow [y_2, y_3, y_4]$



PatchTST: Patching



Motivation: Long input sequences

- The attention matrix has shape $A \in \mathbb{R}^{(T \times T)}$
- Computation and memory scale quadratically $\mathcal{O}(T^2)$

A TIME SERIES IS WORTH 64 WORDS:

LONG-TERM FORECASTING WITH TRANSFORMERS

Yuqi Nie^{1*}, Nam H. Nguyen^{2*}, Phanwadee Sinthong², Jayant Kalagnanam²

¹Princeton University ²IBM Research

ynie@princeton.edu, nnguyen@us.ibm.com, Gift.Sinthong@ibm.com, jayant@us.ibm.com

Patching

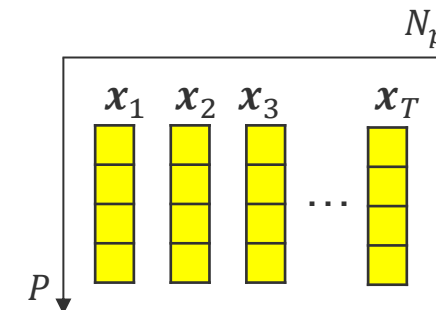
- Divide the time series into patches of P consecutive time steps with stride S
- Each patch is treated as one token
- Reduces the attention complexity to $\mathcal{O}(T^2) \rightarrow \mathcal{O}(N_p^2)$

$$N_p = \left\lfloor \frac{T - P}{S} \right\rfloor + 2$$

PatchTST: Encoder-Only Transformer for time series

BUT:

Patching replaces individual time steps with patch tokens. Where does the channel dimension go?



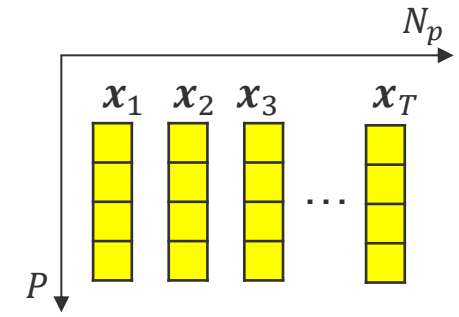
Channel independence vs. Channel mixing



Cyber-Physical Systems
in Mechanical Engineering TU Berlin

Multivariate time series: $X \in \mathbb{R}^{(T \times N)}$ → After patching: $X_{\text{patch}} \in \mathbb{R}^{(C \times N_p \times P)}$

Question: *How should the channel dimension C be incorporated into the Transformer?*



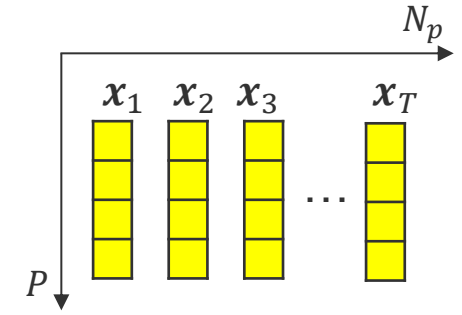
1. **Channel Independence** : Process each channel as a separate patch token sequence
2. **Channel Mixing**: Combine channels within each patch token

Channel independence



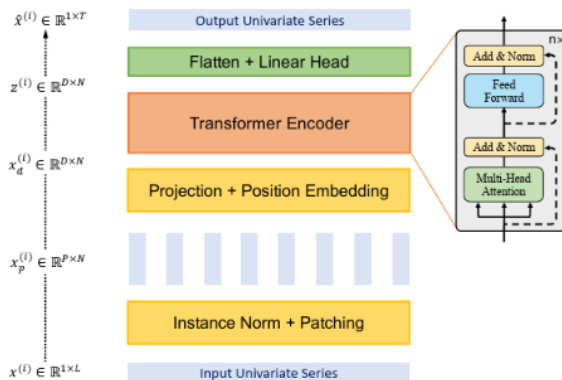
Multivariate time series: $X \in \mathbb{R}^{(T \times N)}$ → After patching: $X_{\text{patch}} \in \mathbb{R}^{(C \times N_p \times P)}$

Question: *How should the channel dimension C be incorporated into the Transformer?*

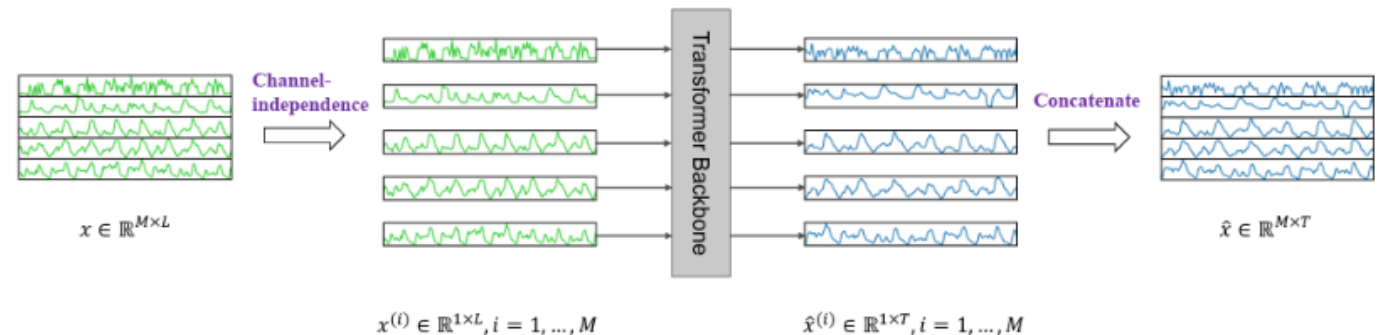


1. Channel Independence

- Treat each channel as a separate sequence of patch tokens
- Merge the channel and batch dimensions: $B \times C \times N_p \times d_{\text{model}} \rightarrow (BC) \times N_p \times d_{\text{model}}$
- Process all channel sequences **independently** but in **parallel**



Y. Nie et al. 2023



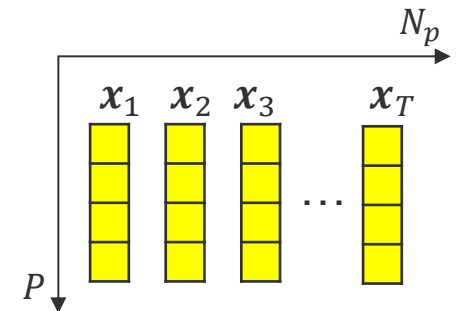
Y. Nie et al. 2023

Channel mixing



Multivariate time series: $X \in \mathbb{R}^{(T \times N)}$ $\longrightarrow$ After patching: $X_{\text{patch}} \in \mathbb{R}^{(C \times N_p \times P)}$

Question: *How should the channel dimension C be incorporated into the Transformer?*



2. Channel Mixing

- Each patch token contains P time steps from C channels: $X_{\text{patch}} \in \mathbb{R}^{B \times N_p \times (CP)}$
- Project each multivariate patch to the model dimension: $B \times N_p \times (CP) \rightarrow B \times N_p \times d_{\text{model}}$
- Process one joint sequence of N_p patch tokens
- Allows the patch embedding to learn cross-channel relationships

Advanced Cross-Channel Modeling:

- Channel-Token Attention $\rightarrow T \times C \rightarrow C \times T \rightarrow$ “*iTransformer*”, Liu et al. 2023
- Factorized Temporal and Channel Attention $\rightarrow$ “*Crossformer*”, Zhang et al. 2023

Transformer for Time Series: Summary



Cyber-Physical Systems
in Mechanical Engineering TU Berlin

- Time-series data can be tokenized in different ways (individual time steps vs. patches)
- The architecture depends on the task
 - **Encoder-only:** classification, regression, direct forecasting
 - **Decoder-only:** causal and autoregressive forecasting
 - **Encoder-Decoder:** sequence-to-sequence prediction conditioned on an input sequence
- Patching enables longer input sequences
 - **Quadratic scaling** with context length! $\rightarrow A = \text{softmax}(S_{\text{scaled}}) \in \mathbb{R}^{(T \times T)}$
 - Reduces the number of tokens from T to N_p
 - Reduces the attention complexity to $\mathcal{O}(T^2) \rightarrow \mathcal{O}(N_p^2)$
 - **Question:** Channel independence vs. Channel mixing



Questions?